Taller 5: Filtrado de Imágenes Digitales

Camilo Eduardo Muñoz Albornoz
Procesamiento Digital de Imágenes
Universidad Tecnológica de Pereira

Código fuente en GitHub

Repositorio: github.com/CamiloMunozAL/ProcDImagenes

Resumen

En este taller se estudiaron diferentes técnicas de filtrado espacial aplicadas a imágenes digitales en escala de grises. Se implementaron filtros paso bajo de promedio, media y mediana, así como filtros paso alto para destacar bordes y variaciones de intensidad.

Los algoritmos fueron desarrollados en Python utilizando ciclos anidados, con apoyo de NumPy, OpenCV y Matplotlib. Se utilizó una fotografía de un paisaje para observar y comparar los efectos producidos por diferentes máscaras.

Finalmente, se evaluó la aplicación de un filtro de mediana antes de un filtro paso alto, con el propósito de observar cómo el suavizado previo influye en la identificación de bordes y detalles de la imagen.

1. Objetivos

Los objetivos del taller fueron:

1. Desarrollar una función basada en ciclos que permita aplicar filtros paso bajo de promedio a una imagen, recibiendo como argumentos la imagen y una máscara. Observar los resultados obtenidos con tres máscaras: promedio uniforme 3×3, promedio uniforme 5×5 y promedio ponderado 3×3.

2. Desarrollar una función basada en ciclos que permita aplicar filtros paso bajo de media de 3×3 y 5×5 a una imagen en escala de grises, recibiendo como entrada la imagen y obteniendo como salida la imagen filtrada.

3. Desarrollar una función basada en ciclos para aplicar filtros paso bajo de mediana de 3×3 y 5×5 a una imagen en escala de grises, observando los resultados obtenidos.

4. Desarrollar una función basada en ciclos que permita aplicar filtros paso alto mediante máscaras de 3×3 y 5×5 a una imagen en escala de grises, analizando el efecto producido sobre los bordes y detalles.

5. Aplicar primero un filtro de mediana a la imagen en escala de grises y posteriormente un filtro paso alto. Comparar este resultado con la aplicación directa del filtro paso alto sobre la imagen original, observando las diferencias en los bordes y detalles resaltados.

2. Fundamentos teóricos

2.1. Filtrado de imágenes digitales

El filtrado de imágenes digitales consiste en aplicar operaciones matemáticas sobre los píxeles de una imagen para modificar determinadas características.

Entre sus principales aplicaciones se encuentran la reducción de ruido, el suavizado, el realce de detalles y la detección de bordes.

El filtrado puede realizarse en el dominio espacial o en el dominio de la frecuencia.

En este taller se trabajó principalmente con **filtrado espacial**, donde las operaciones se realizan directamente sobre los valores de los píxeles y sus vecinos.

2.2. Máscaras y operaciones de vecindad

Una máscara o núcleo es una matriz pequeña que se desplaza sobre la imagen y permite calcular un nuevo valor para cada píxel.

Por ejemplo, una máscara de 3×3 considera el píxel central y los ocho píxeles que lo rodean.

Su estructura general es:

$$W = \begin{bmatrix} w_1 & w_2 & w_3 \\ w_4 & w_5 & w_6 \\ w_7 & w_8 & w_9 \end{bmatrix}$$

En un filtrado lineal, cada valor de la máscara se multiplica por el píxel correspondiente de la vecindad y posteriormente se suman los resultados.

Para una máscara simétrica de 3×3:

$$g(x,y) = \sum_{i=-1}^{1} \sum_{j=-1}^{1} f(x+i, y+j) w(i,j)$$

Donde:

- $f(x,y)$ representa la imagen original.
- $w(i,j)$ representa los coeficientes de la máscara.
- $g(x,y)$ representa el valor obtenido después del filtrado.

El tamaño y los coeficientes de la máscara determinan cómo se modifica la imagen.

2.3. Filtros paso bajo

Los filtros paso bajo atenúan las variaciones rápidas de intensidad entre píxeles vecinos.

Estas variaciones suelen estar relacionadas con bordes, texturas finas y determinados tipos de ruido.

Por esta razón, su aplicación produce un efecto de suavizado o desenfoque.

Entre los filtros paso bajo estudiados se encuentran:

- Filtro de promedio.
- Filtro de media.
- Filtro de mediana.

2.3.1. Filtro de promedio

El filtro de promedio reemplaza cada píxel por una combinación de las intensidades de sus vecinos, utilizando los coeficientes de una máscara.

Una máscara uniforme de 3×3 es:

$$W_1 = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

En este caso todos los píxeles tienen la misma importancia.

También se pueden utilizar máscaras ponderadas, que asignan pesos diferentes a los píxeles.

Por ejemplo:

$$W_3 = \frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

En esta máscara, el píxel central tiene mayor peso que los demás.

2.3.2. Filtro de media

El filtro de media calcula el promedio aritmético de las intensidades de una vecindad.

Su expresión general es:

$$g(x, y) = \frac{1}{N} \sum_{k=1}^{N} P_k$$

Donde N corresponde al número de píxeles de la ventana.

Para una ventana de 3×3:

$$N = 9$$

Mientras que para una ventana de 5×5:

$$N = 25$$

El filtro de media es equivalente a un filtro de promedio con coeficientes uniformes.

2.3.3. Filtro de mediana

El filtro de mediana es un filtro no lineal.

A diferencia del filtro de media, no calcula un promedio aritmético, sino que ordena las intensidades de la vecindad y selecciona el valor central de la lista ordenada.

Por ejemplo:

$$P = \{100, 102, 101, 103, 255, 99, 100, 102, 101\}$$

Después de ordenar:

$$P = \{99, 100, 100, 101, 101, 102, 102, 103, 255\}$$

La mediana es:

$$\text{Mediana}(P) = 101$$

Este procedimiento resulta especialmente útil para reducir ruido impulsivo, como el ruido de sal y pimienta, sin modificar tanto algunos contornos importantes.

2.4. Filtros paso alto

Los filtros paso alto destacan las variaciones rápidas de intensidad presentes en una imagen.
Estas variaciones corresponden principalmente a bordes, detalles finos y texturas.
Una máscara paso alto de 3×3 es:

$$W = \begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Una característica importante es que la suma de sus coeficientes es cero:

$$8 - 8 = 0$$

Esto significa que una región completamente uniforme produce una respuesta igual a cero.
En cambio, cuando existen diferencias importantes entre el píxel central y sus vecinos, la respuesta puede tener una magnitud elevada.
Los resultados pueden ser positivos o negativos, por lo que es necesario utilizar tipos de datos que permitan representar ambos signos.
Para visualizar la magnitud de las respuestas se puede utilizar el valor absoluto:

$$g_{\text{visual}}(x, y) = |g(x, y)|$$

Si se desea representar el resultado en una imagen de 8 bits, los valores deben ajustarse al intervalo de 0 a 255.

2.5. Tratamiento de los bordes

Cuando una máscara se aproxima al límite de una imagen, parte de su vecindad puede quedar por fuera de las dimensiones disponibles.
Este problema se conoce como **problema del contorno**.
En las implementaciones de los filtros paso bajo se decidió procesar solamente los píxeles que permiten utilizar una ventana completa, conservando los bordes originales.
En el filtro paso alto se inicializó la imagen de salida con ceros, por lo que las posiciones exteriores que no fueron procesadas permanecieron en cero.

3. Desarrollo y resultados

3.1. Preparación de la imagen

Para todos los ejercicios se utilizó una fotografía de un paisaje que contiene montañas, árboles, vegetación, cielo y un lago con reflejos.
Esta imagen permite observar el comportamiento de los filtros sobre regiones uniformes, bordes y zonas con diferentes texturas.
Se cargó directamente en escala de grises utilizando OpenCV.

```
1 import cv2
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 imagen = cv2.imread(
```

```
    'images/paisaje.png',
    cv2.IMREAD_GRAYSCALE
)

print("Tamano de la imagen:", imagen.shape)
```

La función `cv2.IMREAD_GRAYSCALE` permite cargar la fotografía utilizando un único canal de intensidad.

Los píxeles se representan normalmente mediante valores enteros entre 0 y 255.

3.2. Punto 1: Filtros paso bajo de promedio

Objetivo

Implementar mediante ciclos una función que reciba una imagen y una máscara, y devuelva la imagen filtrada. Aplicar tres máscaras de promedio y analizar sus efectos sobre la imagen.

Implementación

Se definieron las tres máscaras solicitadas:

$$W_1 = \frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$$W_2 = \frac{1}{25}\begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

$$W_3 = \frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

En Python:

```
mascara1 = np.ones((3, 3)) / 9

mascara2 = np.ones((5, 5)) / 25

mascara3 = np.array([
    [1, 2, 1],
    [2, 4, 2],
    [1, 2, 1]
]) / 16
```

Posteriormente se desarrolló una función que recorre la imagen y la máscara mediante cuatro ciclos anidados.

```
def filtro_promedio(imagen, mascara):

    # Dimensiones de la imagen
    filas_imagen, columnas_imagen = imagen.shape

    # Dimensiones de la mascara
    filas_mascara, columnas_mascara = mascara.shape

```

```
    # Extension de la mascara
    extension_fila = filas_mascara // 2
    extension_columna = columnas_mascara // 2

    # Copia de la imagen original
    imagen_filtrada = imagen.astype(
        np.float64
    ).copy()

    # Recorrer la imagen
    for fila_imagen in range(
        extension_fila,
        filas_imagen - extension_fila
    ):
        for columna_imagen in range(
            extension_columna,
            columnas_imagen - extension_columna
        ):

            suma = 0

            # Recorrer la mascara
            for fila_mascara in range(filas_mascara):
                for columna_mascara in range(columnas_mascara):

                    pixel = imagen[
                        fila_imagen + fila_mascara
                        - extension_fila,

                        columna_imagen + columna_mascara
                        - extension_columna
                    ]

                    peso = mascara[
                        fila_mascara,
                        columna_mascara
                    ]

                    suma += pixel * peso

            imagen_filtrada[
                fila_imagen,
                columna_imagen
            ] = suma

    return np.clip(
        imagen_filtrada, 0, 255
    ).astype(np.uint8)
```

Los dos primeros ciclos recorren las posiciones de la imagen y los dos últimos recorren los elementos de la máscara.

Cada píxel se multiplica por su peso correspondiente y los resultados se suman para obtener la nueva intensidad.

Se utilizó una imagen de salida independiente para evitar que los píxeles ya filtrados modificaran los siguientes cálculos.

Aplicación

```
imagen_filtrada1 = filtro_promedio(
```

```
    imagen, mascara1
)

imagen_filtrada2 = filtro_promedio(
    imagen, mascara2
)

imagen_filtrada3 = filtro_promedio(
    imagen, mascara3
)

imagenes = [
    imagen,
    imagen_filtrada1,
    imagen_filtrada2,
    imagen_filtrada3
]

titulos = [
    'Imagen Original',
    'Filtro Promedio 3x3',
    'Filtro Promedio 5x5',
    'Filtro Ponderado 3x3'
]

fig, axs = plt.subplots(2, 2, figsize=(12, 8))

for a, img, titulo in zip(
    axs.flat, imagenes, titulos
):
    a.imshow(
        img,
        cmap='gray',
        vmin=0,
        vmax=255
    )
    a.set_title(titulo)
    a.axis('off')

plt.tight_layout()

plt.savefig(
    'results/resultado_filtro_promedio.png'
)

plt.show()
```

Los resultados se presentan en la Figura 1.

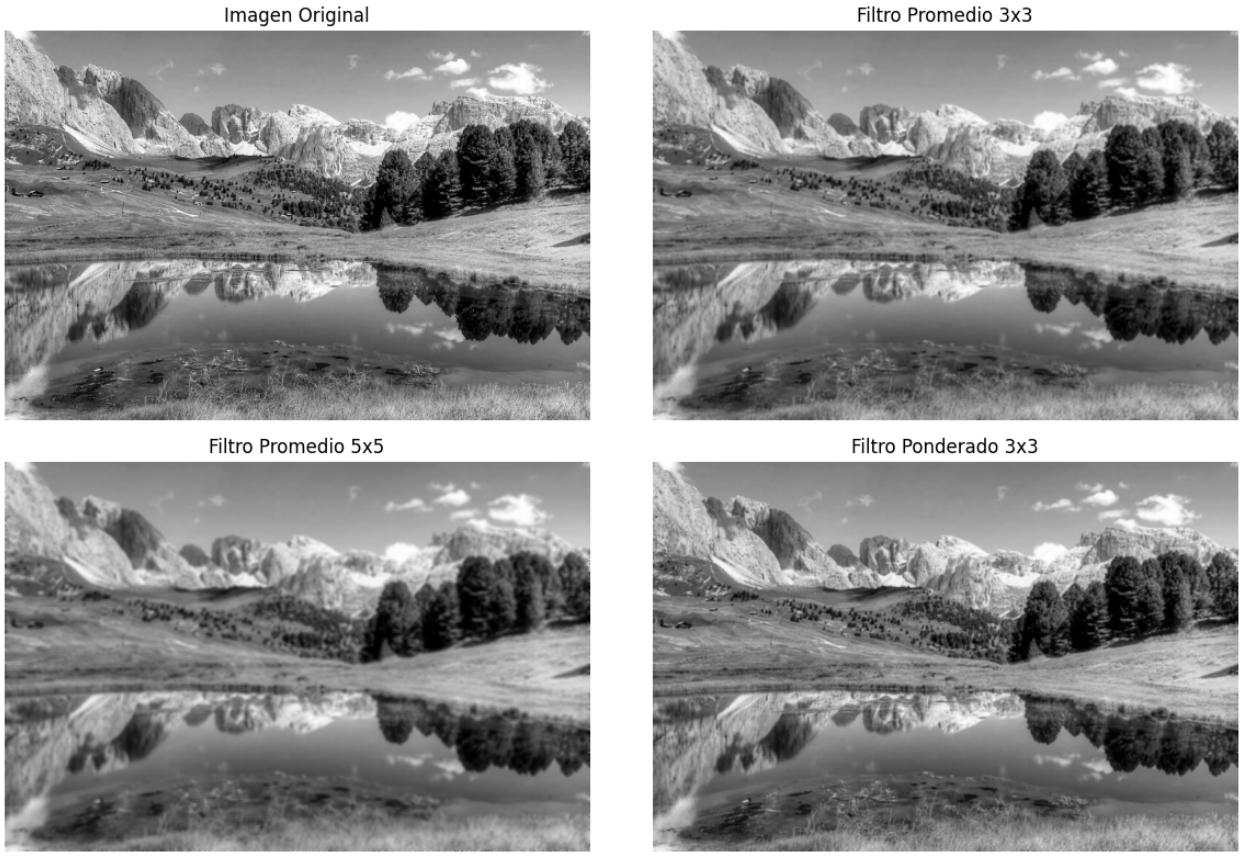


Figura 1: Comparación de la imagen original con los filtros de promedio uniforme y ponderado.

Análisis de resultados

Al comparar los resultados, se observa que la máscara ponderada de 3×3 produce el menor suavizado, conservando mejor los detalles y bordes de las montañas, los árboles y sus reflejos en el agua.

Por otra parte, la máscara uniforme de 3×3 genera un suavizado mayor, haciendo que los bordes de los árboles y las montañas pierdan algo más de definición.

Finalmente, la máscara de 5×5 produce el desenfoque más evidente, con una mayor pérdida de detalles.

Estas diferencias se deben tanto al tamaño de la máscara como a la distribución de sus coeficientes.

En la máscara ponderada, el píxel central tiene mayor importancia, permitiendo conservar mejor algunas características de la imagen original.

En conclusión, los filtros paso bajo reducen las variaciones de intensidad entre píxeles vecinos, produciendo un efecto de suavizado.

Sin embargo, también pueden provocar una pérdida de detalles y definición de los bordes.

En este caso, el filtro ponderado de 3×3 ofreció el mejor equilibrio visual entre suavizado y conservación de detalles.

3.3. Punto 2: Filtros paso bajo de media

Objetivo

Desarrollar mediante ciclos una función que permita aplicar filtros de media de 3×3 y 5×5 a una imagen en escala de grises.

Implementación

En este punto se calculó directamente el promedio aritmético de los píxeles de cada vecindad. Para una ventana cuadrada de tamaño n, la operación es:

$$g(x,y) = \frac{1}{n^2} \sum_{i=-r}^{r} \sum_{j=-r}^{r} f(x+i, y+j)$$

Donde:

$$r = \left\lfloor \frac{n}{2} \right\rfloor$$

La función desarrollada fue:

```python
def filtro_media(imagen, tamano):

    if tamano % 2 == 0 or tamano < 1:
        raise ValueError(
            "El tamano debe ser impar y mayor que 0"
        )

    # Dimensiones de la imagen
    filas_imagen, columnas_imagen = imagen.shape

    # Extension de la ventana
    extension = tamano // 2

    # Copia de la imagen original
    imagen_filtrada = imagen.astype(
        np.float64
    ).copy()

    # Recorrer cada pixel
    for fila_imagen in range(
        extension,
        filas_imagen - extension
    ):
        for columna_imagen in range(
            extension,
            columnas_imagen - extension
        ):

            suma = 0

            # Recorrer la vecindad
            for i in range(
                -extension,
                extension + 1
            ):
                for j in range(
                    -extension,
                    extension + 1
                ):

                    pixel = int(imagen[
                        fila_imagen + i,
                        columna_imagen + j
                    ])

                    suma += pixel
```

```
47
48              # Calcular la media
49              imagen_filtrada[
50                  fila_imagen,
51                  columna_imagen
52              ] = suma / (tamano * tamano)
53
54      return np.clip(
55          imagen_filtrada, 0, 255
56      ).astype(np.uint8)
```

A diferencia del punto anterior, en este procedimiento no es necesario multiplicar cada píxel por un coeficiente de una máscara.

Simplemente se suman las intensidades y se dividen entre el número de píxeles.

Además, se utilizó `int()` al obtener cada píxel para evitar desbordamientos durante la acumulación de valores `uint8`.

Aplicación

```
1  imagen_media3 = filtro_media(imagen, 3)
2
3  imagen_media5 = filtro_media(imagen, 5)
4
5  imagenes = [
6      imagen,
7      imagen_media3,
8      imagen_media5
9  ]
10
11 titulos = [
12     "Imagen Original",
13     "Filtro Media 3x3",
14     "Filtro Media 5x5"
15 ]
16
17 fig, axs = plt.subplots(1, 3, figsize=(15, 5))
18
19 for ax, img, titulo in zip(
20     axs, imagenes, titulos
21 ):
22     ax.imshow(
23         img,
24         cmap="gray",
25         vmin=0,
26         vmax=255
27     )
28     ax.set_title(titulo)
29     ax.axis("off")
30
31 plt.tight_layout()
32
33 plt.savefig(
34     "results/resultado_filtro_media.png"
35 )
36
37 plt.show()
```

El resultado se observa en la Figura 2.



Figura 2: Comparación de la imagen original con filtros de media de 3×3 y 5×5.

Análisis de resultados

Al observar las imágenes, se evidencia que el filtro de media 3×3 produce un suavizado ligero, conservando gran parte de los detalles de las montañas, los árboles y sus reflejos en el agua.

Por otro lado, el filtro de media 5×5 genera un desenfoque mayor, haciendo que los bordes de las montañas, los árboles y algunos detalles de la vegetación pierdan más definición.

En conclusión, los filtros de media permiten suavizar una imagen al reducir las variaciones de intensidad entre píxeles vecinos.

Se observa que, al aumentar el tamaño de la ventana, el suavizado es mayor, pero también se pierde más información de los detalles y bordes.

3.4. Punto 3: Filtros paso bajo de mediana

Objetivo

Implementar mediante ciclos filtros de mediana de 3×3 y 5×5 y observar su efecto sobre una imagen en escala de grises.

Implementación

El filtro de mediana obtiene los valores de cada vecindad, los ordena de menor a mayor y selecciona el valor central.

Para una ventana de 3×3 se ordenan nueve valores y se selecciona el quinto elemento.

Para una ventana de 5×5 se ordenan 25 valores y se selecciona el elemento número 13.

Debido a que Python utiliza índices desde cero, las posiciones corresponden a 4 y 12 respectivamente.

La función desarrollada fue:

```
1  def filtro_mediana(imagen, tamano):
2
3      if tamano % 2 == 0 or tamano < 1:
4          raise ValueError(
5              "El tamano de la mascara debe ser impar"
6          )
7
8      # Dimensiones de la imagen
9      filas_imagen, columnas_imagen = imagen.shape
10
11     # Extension de la ventana
12     extension = tamano // 2
```

```
13
14      # Copia de la imagen original
15      imagen_filtrada = imagen.copy()
16
17      # Recorrer cada pixel
18      for fila_imagen in range(
19          extension,
20          filas_imagen - extension
21      ):
22          for columna_imagen in range(
23              extension,
24              columnas_imagen - extension
25          ):
26
27              # Extraer vecinos
28              vecinos = []
29
30              for i in range(
31                  -extension,
32                  extension + 1
33              ):
34                  for j in range(
35                      -extension,
36                      extension + 1
37                  ):
38
39                      pixel = int(imagen[
40                          fila_imagen + i,
41                          columna_imagen + j
42                      ])
43
44                      vecinos.append(pixel)
45
46              # Ordenar los vecinos
47              vecinos.sort()
48
49              # Obtener la mediana
50              mediana = vecinos[
51                  len(vecinos) // 2
52              ]
53
54              imagen_filtrada[
55                  fila_imagen,
56                  columna_imagen
57              ] = mediana
58
59      return imagen_filtrada
```

La lista `vecinos` almacena las intensidades de cada ventana.
Posteriormente, el método `sort()` ordena los valores y se obtiene el elemento central.
De esta manera, el nuevo valor del píxel corresponde a una intensidad existente dentro de su vecindad.

Aplicación

```
1  imagen_mediana3 = filtro_mediana(imagen, 3)
2
3  imagen_mediana5 = filtro_mediana(imagen, 5)
4
5  imagenes = [
6      imagen,
```

```
    imagen_mediana3,
    imagen_mediana5
]

titulos = [
    "Imagen Original",
    "Filtro Mediana 3x3",
    "Filtro Mediana 5x5"
]

fig, axs = plt.subplots(1, 3, figsize=(15, 5))

for ax, img, titulo in zip(
    axs, imagenes, titulos
):
    ax.imshow(
        img,
        cmap="gray",
        vmin=0,
        vmax=255
    )
    ax.set_title(titulo)
    ax.axis("off")

plt.tight_layout()

plt.savefig(
    "results/resultado_filtro_mediana.png"
)

plt.show()
```

Los resultados se muestran en la Figura 3.



Figura 3: Aplicación de filtros de mediana de 3×3 y 5×5.

Análisis de resultados

Al observar los resultados, se evidencia que el filtro de mediana 3×3 produce cambios leves en la imagen, conservando gran parte de los detalles y bordes de las montañas, los árboles y sus reflejos en el agua.

Por otra parte, el filtro de mediana 5×5 genera un suavizado mayor, eliminando algunos detalles pequeños de la vegetación y las texturas, pero manteniendo relativamente bien los contornos principales

de los objetos.

Las diferencias respecto a la imagen original no son muy pronunciadas, debido a que la fotografía no presenta una cantidad evidente de ruido impulsivo.

En conclusión, el filtro de mediana permite reducir valores de intensidad aislados y conservar relativamente bien los bordes.

Al aumentar el tamaño de la ventana se eliminan más detalles pequeños, por lo que es importante seleccionar el tamaño según las características de la imagen.

3.5. Punto 4: Filtros paso alto

Objetivo

Desarrollar una función basada en ciclos que permita aplicar filtros paso alto mediante máscaras de 3×3 y 5×5, observando el realce de cambios de intensidad y bordes.

Implementación

Para el filtro paso alto 3×3 se utilizó la máscara:

$$W_1 = \begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Para la máscara 5×5 se asignó un valor de -1 a las 24 posiciones periféricas y un valor de 24 al centro:

$$W_2 = \begin{bmatrix} -1 & -1 & -1 & -1 & -1 \\ -1 & -1 & -1 & -1 & -1 \\ -1 & -1 & 24 & -1 & -1 \\ -1 & -1 & -1 & -1 & -1 \\ -1 & -1 & -1 & -1 & -1 \end{bmatrix}$$

Ambas máscaras tienen una suma de coeficientes igual a cero.

Su implementación en Python fue:

```
mascara_alto3 = np.array([
    [-1, -1, -1],
    [-1, 8, -1],
    [-1, -1, -1]
])

mascara_alto5 = -np.ones((5, 5))

mascara_alto5[2, 2] = 24
```

Posteriormente se construyó la función para aplicar las máscaras mediante ciclos anidados.

```
def filtro_paso_alto(imagen, mascara):

    # Dimensiones de la imagen
    filas_imagen, columnas_imagen = imagen.shape

    # Dimensiones de la mascara
    filas_mascara, columnas_mascara = mascara.shape

    # Extension de la mascara
    extension_fila = filas_mascara // 2
```

```
    extension_columna = columnas_mascara // 2

    # Imagen de salida en float64
    imagen_filtrada = np.zeros_like(
        imagen,
        dtype=np.float64
    )

    # Recorrer cada pixel
    for fila_imagen in range(
        extension_fila,
        filas_imagen - extension_fila
    ):
        for columna_imagen in range(
            extension_columna,
            columnas_imagen - extension_columna
        ):

            suma = 0

            # Recorrer la mascara
            for fila_mascara in range(filas_mascara):
                for columna_mascara in range(columnas_mascara):

                    pixel = float(imagen[
                        fila_imagen + fila_mascara
                        - extension_fila,

                        columna_imagen + columna_mascara
                        - extension_columna
                    ])

                    peso = mascara[
                        fila_mascara,
                        columna_mascara
                    ]

                    suma += pixel * peso

            imagen_filtrada[
                fila_imagen,
                columna_imagen
            ] = suma

    return imagen_filtrada
```

La imagen de salida se inicializó con valores cero y tipo `float64`, debido a que los filtros paso alto pueden generar respuestas negativas y superiores a 255.

A diferencia de los filtros paso bajo anteriores, no se convierten inmediatamente los resultados a `uint8`, con el fin de conservar la información de los cálculos.

Aplicación y visualización

```
imagen_alto3 = filtro_paso_alto(
    imagen, mascara_alto3
)

imagen_alto5 = filtro_paso_alto(
    imagen, mascara_alto5
```

```
)

# Magnitud absoluta de las respuestas
imagen_alto3_visual = np.clip(
    np.abs(imagen_alto3),
    0,
    255
).astype(np.uint8)

imagen_alto5_visual = np.clip(
    np.abs(imagen_alto5),
    0,
    255
).astype(np.uint8)

imagenes = [
    imagen,
    imagen_alto3_visual,
    imagen_alto5_visual
]

titulos = [
    "Imagen Original",
    "Filtro Paso Alto 3x3",
    "Filtro Paso Alto 5x5"
]

fig, axs = plt.subplots(1, 3, figsize=(15, 5))

for ax, img, titulo in zip(
    axs, imagenes, titulos
):
    ax.imshow(
        img,
        cmap="gray",
        vmin=0,
        vmax=255
    )
    ax.set_title(titulo)
    ax.axis("off")

plt.tight_layout()

plt.savefig(
    "results/resultado_filtro_paso_alto.png"
)

plt.show()
```

Los resultados se presentan en la Figura 4.

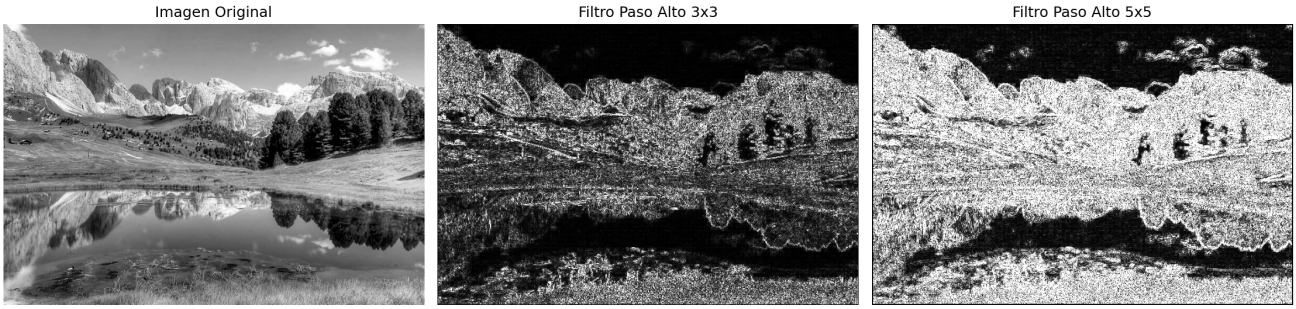


Figura 4: Comparación de la imagen original con filtros paso alto de 3×3 y 5×5.

Análisis de resultados

Al observar las imágenes, se evidencia que el filtro paso alto 3×3 destaca los bordes de las montañas, los árboles y algunas texturas de la vegetación.

Por otra parte, las regiones más uniformes, como el cielo y algunas zonas del agua, aparecen oscuras debido a que presentan pocas variaciones de intensidad.

En el filtro paso alto 5×5 se observa una respuesta mucho más intensa, especialmente en las montañas, los árboles y el pasto.

Esto se debe a que la máscara considera una vecindad mayor y utiliza coeficientes que pueden generar valores de mayor magnitud.

También se observa que algunas regiones aparecen casi completamente blancas, debido a que los valores que superan 255 se limitan a ese máximo durante la visualización.

Análisis de saturación

Para comprender mejor este comportamiento, se calcularon las respuestas máximas absolutas de ambos filtros y la cantidad de píxeles que superaron el valor 255.

El código utilizado fue:

```
print(
    "Respuesta maxima absoluta 3x3:",
    np.abs(imagen_alto3).max()
)

print(
    "Respuesta maxima absoluta 5x5:",
    np.abs(imagen_alto5).max()
)

print(
    "Pixeles saturados 3x3:",
    np.count_nonzero(
        np.abs(imagen_alto3) > 255
    )
)

print(
    "Pixeles saturados 5x5:",
    np.count_nonzero(
```

```
        np.abs(imagen_alto5) > 255
    )
)
```

Los valores obtenidos fueron:

- Respuesta máxima absoluta 3×3: 1214.
- Respuesta máxima absoluta 5×5: 3802.
- Píxeles con magnitud superior a 255, filtro 3×3: 21808.
- Píxeles con magnitud superior a 255, filtro 5×5: 152985.

Estos resultados muestran que el filtro 5×5 genera respuestas considerablemente mayores que el filtro 3×3.

Por esta razón, al utilizar:

```
np.clip(np.abs(imagen_alto5), 0, 255)
```

muchos valores quedan representados con intensidad 255.

Esto explica la presencia de grandes regiones blancas en la imagen filtrada.

Sin embargo, una mayor intensidad visual no significa necesariamente una mejor detección de bordes, ya que también depende de los coeficientes de la máscara y del tratamiento de los resultados.

En conclusión, los filtros paso alto permiten destacar cambios bruscos de intensidad, haciendo visibles bordes y texturas.

También se comprobó que es importante considerar la magnitud de las respuestas y la saturación al visualizar los resultados.

3.6. Punto 5: Filtro de mediana seguido de paso alto

Objetivo

Aplicar un filtro de mediana a la imagen original y posteriormente un filtro paso alto.

Comparar el resultado con la aplicación directa del filtro paso alto sobre la imagen, analizando las diferencias en los bordes y detalles resaltados.

Implementación

En este punto se utilizaron las funciones desarrolladas anteriormente.

Se realizaron dos procedimientos:

1. Aplicar un filtro de mediana 3×3 y posteriormente un filtro paso alto 3×3.
2. Aplicar directamente un filtro paso alto 3×3 sobre la imagen original.

La finalidad fue observar cómo el filtrado de mediana previo modifica la respuesta del filtro paso alto.

El código utilizado fue:

```
# Procedimiento 1:
# Mediana seguida de paso alto

imagen_mediana = filtro_mediana(
    imagen, 3
)

resultado_combinado = filtro_paso_alto(
```

```
    imagen_mediana,
    mascara_alto3
)

# Procedimiento 2:
# Paso alto directo

resultado_directo = filtro_paso_alto(
    imagen,
    mascara_alto3
)

# Preparar visualizaciones
combinado_visual = np.clip(
    np.abs(resultado_combinado),
    0,
    255
).astype(np.uint8)

directo_visual = np.clip(
    np.abs(resultado_directo),
    0,
    255
).astype(np.uint8)

# Mostrar resultados
imagenes = [
    imagen,
    combinado_visual,
    directo_visual
]

titulos = [
    "Imagen Original",
    "Mediana 3x3 + Paso Alto 3x3",
    "Paso Alto 3x3 Directo"
]

fig, axs = plt.subplots(1, 3, figsize=(15, 5))

for ax, img, titulo in zip(
    axs, imagenes, titulos
):
    ax.imshow(
        img,
        cmap="gray",
        vmin=0,
        vmax=255
    )
    ax.set_title(titulo)
    ax.axis("off")

plt.tight_layout()

plt.savefig(
    "results/resultado_mediana_paso_alto.png"
)

plt.show()
```

La comparación se presenta en la Figura 5.

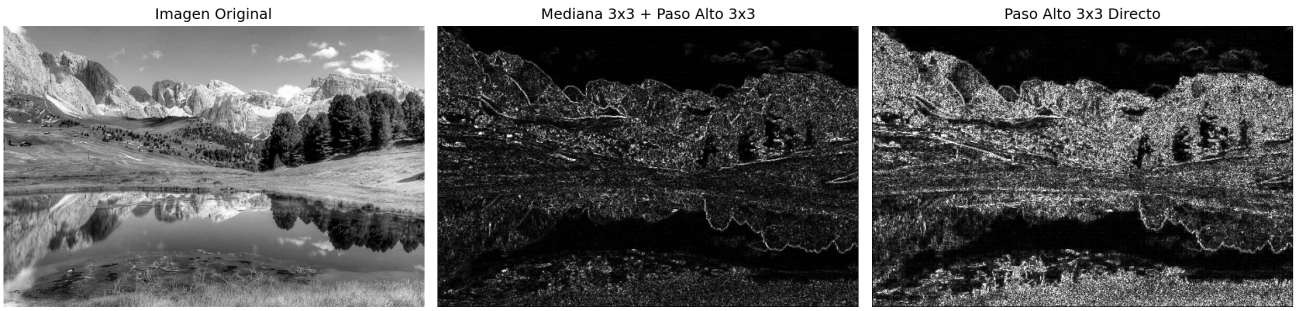


Figura 5: Comparación entre el filtrado de mediana seguido de paso alto y la aplicación directa del paso alto.

Análisis de resultados

Al observar las imágenes, se evidencia que la combinación de mediana y paso alto produce una imagen más oscura, con menos detalles pequeños resaltados, pero en la que se distinguen con mayor claridad algunos bordes principales.

Esto se observa especialmente en los contornos de las montañas, los árboles y sus reflejos en el agua.

Por otra parte, al aplicar directamente el filtro paso alto, se resaltan muchas más variaciones de intensidad, especialmente en la vegetación, el pasto y las texturas de las montañas.

Esto genera una imagen con mayor cantidad de puntos y regiones brillantes, lo que dificulta distinguir algunos de los contornos principales.

La diferencia se debe a que el filtro de mediana reduce algunas variaciones pequeñas de intensidad antes de aplicar el filtro paso alto.

De esta manera, el paso alto posterior resalta menos texturas y ciertos bordes importantes pueden identificarse visualmente con mayor facilidad.

Es importante aclarar que el filtro de mediana no aumenta directamente la intensidad de los bordes.

Su efecto consiste en reducir parte de las variaciones locales, permitiendo que algunos contornos sobresalgan mejor respecto a su entorno.

En conclusión, aplicar primero un filtro de mediana y después uno paso alto permite reducir detalles pequeños y mejorar visualmente la identificación de ciertos bordes principales.

Sin embargo, también puede eliminar información fina de la imagen, por lo que la elección del procedimiento depende del objetivo del procesamiento.

4. Análisis general

El desarrollo del taller permitió comprobar que el filtrado espacial modifica los valores de los píxeles teniendo en cuenta la información de su vecindad.

En los primeros ejercicios se observó que los filtros paso bajo producen un efecto de suavizado.

El tamaño de la máscara y la distribución de sus coeficientes influyen directamente en la cantidad de detalles que se conservan.

La máscara ponderada 3×3 permitió conservar mejor algunos bordes del paisaje, mientras que las ventanas uniformes más grandes produjeron un desenfoque mayor.

El filtro de media mostró que es posible suavizar una imagen calculando directamente el promedio de los valores vecinos.

Por su parte, el filtro de mediana permitió aplicar una operación no lineal, que reemplaza cada píxel por el valor central de su vecindad ordenada.

En la fotografía utilizada, la mediana produjo cambios relativamente leves y conservó buena parte de los contornos principales.

Posteriormente, los filtros paso alto mostraron un comportamiento diferente: en lugar de suavizar las variaciones de intensidad, las destacaron.

Esto permitió identificar bordes de montañas, árboles, vegetación y reflejos sobre el agua.

También se observó que las máscaras paso alto pueden producir respuestas negativas o superiores a 255.

Los resultados del punto 4 demostraron que una gran cantidad de píxeles puede alcanzar el valor máximo durante la visualización, provocando saturación.

Finalmente, el punto 5 permitió comprobar que combinar filtros puede modificar significativamente los resultados.

La aplicación previa de mediana redujo algunas texturas y variaciones pequeñas antes del filtro paso alto.

Como consecuencia, varios contornos principales se distinguieron mejor que en el resultado del filtrado paso alto directo.

En conjunto, los ejercicios mostraron que no existe un filtro universalmente mejor, sino que su utilidad depende de las características de la imagen y del objetivo del procesamiento.

5. Conclusiones

Los filtros espaciales permiten modificar una imagen digital mediante operaciones sobre los píxeles y sus vecinos. Su implementación con ciclos facilitó comprender el funcionamiento de las máscaras y el cálculo de los nuevos valores.

Los filtros paso bajo de promedio y media permitieron observar que una ventana de mayor tamaño genera generalmente un suavizado más intenso, aunque también puede producir una mayor pérdida de detalles.

El filtro de promedio ponderado conservó mejor ciertos bordes de la fotografía que el promedio uniforme del mismo tamaño, debido a la mayor importancia asignada al píxel central y a sus vecinos cercanos.

El filtro de mediana permitió comprender una forma diferente de procesar la vecindad, utilizando el valor central de una lista ordenada. En las pruebas realizadas conservó relativamente bien los contornos principales, aunque también redujo algunos detalles finos.

Los filtros paso alto permitieron destacar cambios bruscos de intensidad y visualizar bordes y texturas presentes en el paisaje.

Además, se comprobó la importancia de trabajar con tipos de datos adecuados, ya que las operaciones pueden producir valores negativos o superiores al rango de una imagen de 8 bits.

La comparación final mostró que aplicar un filtro de mediana antes del paso alto reduce algunas variaciones pequeñas y facilita la identificación visual de ciertos bordes importantes.

En general, el taller permitió comprender la relación entre **máscaras, vecindad, suavizado, mediana y detección de bordes**, así como la importancia de seleccionar las técnicas de filtrado según el resultado que se desea obtener.